CSD 302 Design and Analysis of Algorithms

Final, Spring 2019- 20, Max 100, Submit by 18:00 hrs, June 7, 2020

Name _____ **Roll No.** _____ **Dept** _____

Note (i) You can quote any result covered in the lectures without proof but any other claim should be formally justified.
(iii) Negative 1 for incorrect answers to the Yes/No True/False questions.

Part A

1. Fill in the blanks or mark the correct answer **(3 × 10)**

   (a) Consider the following recurrence that came from an integer sorting algorithm

   $$T(n,b) = T(n,b/2) + O(n) \qquad T(n,1) = O(n)$$

   Then the recurrence has solution $T(n,b) =$ $\underline{O(n \log b)}$

   (b) In an instance of the UNION-FIND problem involving n elements, suppose all the k union operations occur before all the m *find* operations. Here $k \leq n-1 \leq m$.
   Then we can design an algorithm to process this sequence of operations in $\underline{O(m+k)}$ time (should be the best possible).
   We will use the path-compression heuristic, so that no node (other than the root) can be charged more than once. There can be multiple roots depending on the number of subsets otherwise the problem is trivial for one subset If cost for initialization is added, then $O(m+n)$.

   (c) Let T be a rooted tree (not necessarily binary), with unit edge capacities. Given two arbitrary distinct vertices $s, t \in T$, the s-t mincut equals $\underline{1}$.
   There is a unique path between s and t and consider the cut that consists of the two subtrees formed by removing any edge in that path.

   (d) Given a graph G, the matrix $D(G)$ is defined as follows - $D[i,j] 1 \leq i,j \leq |V| = e \in E$ such that e is the first edge of the shortest path between V_i and V_j. Then we can find the shortest path between any pair of vertices V_s and V_t in

   i. $O(n^2)$ steps
   ii. $O(n)$ steps
   iii. $O(\ell)$ steps where ℓ is the number of edges in the shortest paths $\surd$ By repeated looking up the first edge (u,v) between x and V_j where initially $x = V_i$ and then $x = v$ until $x = V_j$.
   iv. $O(|E|)$ steps

   (e) Suppose a hash family is defined as $h_a(x) = (C \cdot x + a) \mod N \mod m$ where $a \in \{0, 1, 2, \ldots N-1\}$ and C is a fixed integer in the range $[1 \ldots N-1]$. Here N and m are sizes of the universe and the hash table respectively. Then

   i. h_a is **universal**.
   ii. h_a is **universal** for some values of C.
   iii. h_a is not **universal**. $\surd$ since $h_a(x) = h_a(y)$ for all a when $y - x \mod N = 0$.

   (f) While constructing a skip list, instead of 0.5, suppose we push an element to the next level (independently) with probability $\frac{1}{\log n}$ where n is the maximum number of elements in the skip list at any given time. Then the expected search time and expected space are respectively

   $\underline{O(\frac{\log^2 n}{\log \log n})}$ and $\underline{O(n)}$. $\qquad O(\log n)$ per level for $\frac{\log n}{\log \log n}$ levels.

   (g) If we desire a success probability of $1-q$ for string-matching algorithm using Rabin-Karp method, then we should choose the random modulus as (a function of q and n, the pattern length) $\underline{\frac{\log n \log \log n \log(1/q)}{q}}$.

(h) Professor *Crazy* has shown a lower bound of $\Omega(n^{\log^2 n})$ for solving the 3-SAT problem. What can we deduce for the *Vertex Cover* (VC) problem ?

i. VC cannot be solved faster than $\Omega(n^{\log^2 n})$ $\surd$ since 3-SAT $\leq_{polytime}$ VC, so it will contradict otherwise.

ii. VC is not NP Complete

iii. There may still be a polynomial time algorithm for VC.

iv. There cannot be a polynomial time reduction between VC to 3-SAT.

(i) Given a set of points in the plane, we want to build a data structure to support query of the following kind. Given a **T**-shaped polygon, we want to count the number of points inside the polygon in $O(\log^2 n)$ time. This can be done in

$\underline{O(n \log n)}$ space and $\underline{O(n \log n)}$ preprocessing time.

You can decompose the T-polygon as a disjoint union of three axis parallel rectangles and run three range search queries.

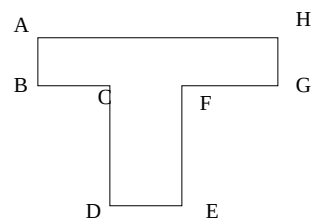

Figure 1: The polygon is described by coordinates A, B , .. and the edges are axes parallel

(j) Find the failure function of KMP string matching algorithm corresponding to the following string. **1000100101** $f(i) = 0000123121$.

Part B

Note : **Problem 2 is compulsory. Do any 4 of remaining 6 problems. All questions do not carry the same marks.**
Restrict your answers to a maximum of 150 words for each question.
You can quote any result covered in the lectures without proof but any other claim should be formally justified.
For any Dynamic Programming solution, you must begin with a clear recurrence - no credit otherwise.

Sometimes algorithms were presented as codes without any explanation or elaboration of the underlying algorithmic ideas. These answers were not considered as was stated clearly under general directives

2. We covered in the lectures that the 3-SAT problem is NP complete. Consider the 4-SAT problem, which is the satisfiability problem for CNF where each clause has exactly 4 literals. **(5+5)**
(a) Is 4-SAT NP Complete ?
Remark Address the two parts independently and do not try to use the result of one part to prove the other. 4-SAT is in NP. We will reduce 3-SAT to 4-SAT in polynomial time - $3-SAT \leq_{poly} 4-SAT$ as follows. For a clause $(x_{i,1} \vee x_{i,2} \vee x_{i,3})$, we will replace with $(x_{i,1} \vee x_{i,2} \vee x_{i,3} \vee y_i)$ and $(x_{i,1} \vee x_{i,2} \vee x_{i,3} \vee \bar{y_i})$ where y_i is a new boolean variable distinct from the x variables. The 4-SAT formula is a conjunction of these 2m clauses where m is the original number of clauses in the 3-SAT formula. Denote the original clause as F and the 4-SAT formula as F'.

This can be done in polynomial time as we are introducing at most m new variables and atmost doubles the number of clauses. We can argue that the $F \iff F'$ as follows. Since y_i is false in one of the two new clauses, one of the x variables must be true to make the new formula satisfiable.

(b) Suppose 3-SAT was not known to be NP Complete but 4-SAT was known to be NPC. Can you prove that 3-SAT is NPC ?

We will now reduce $4 - SAT \leq 3 - SAT$ in the following manner - Let the i-th 4-SAT clause C_i be $(x_{i,1} \vee x_{i,2} \vee x_{i,3} \vee x_{i,4})$. We construct clauses $C'_i = (x_{i,1} \vee x_{i,2} \vee y_i)$ and $C"_i = (x_{i,1} \vee x_{i,2} \vee \bar{y_i})$. Then it follows that $C_i \iff C'_i \wedge C"_i$. This construction is also polynomial time.

Some marks were deducted if any of the following features were missing from the answer - problem in NP , the polynomial time reduction, and both directions of equivalence in reduction

Do any 4 of remaining 6 problems. Note: All questions do not carry the same marks.

3. Given two strings S_1 and S_2 having n symbols each, find out if S_1 and S_2 are cyclic shifts of eachother. For example, *bcacbb* and *acbbbc* are cyclic shifts but *cbcbbb* is not. Your algorithm should work in linear time. **(10)**

Construct the string $S = S_1 \cdot S_1$ (concatenation with itself). Then check is S_2 is a substring of S using any of the string matching algorithms.

Correctness of this approach follows from the observation that $S_1 \cdot S_1$ contains the cyclic shifts $i \;\; 1 \leq n$ starting from position i. Therefore, if S_2 is a cyclic shift iff S_2 is a pattern in $S_1 \cdot S_1$.

Marks were deducted for not giving a complete reasoning why this method works correctly

4. Consider the following family of hash functions $\mathcal{H}$ that satisfies the following property

$$\forall_{x \neq y} \Pr[h(x) = i \wedge h(y) = j] = \frac{1}{m^2} \quad i, j \in [0, m-1] \text{ for a random choice of } h \in \mathcal{H}$$

Here m is the size of the table. What is the expected number of collisions in any location, if we hash an arbitrary set S of n keys. **(10)**

We can repeat the same calculation as we had done for universal hash function by finding the expected number of conflicts with some fixed element x. Let X_i be a 0-1 r.v. which is 1 iff $h(x) = h(x_i)$ for $x \neq x_i$ and $S = \{x_1, x_2 \ldots x_n\}$. Then number of conflicts with x equals $\sum_i X_i$. We want to compute $E[\sum_i X_i]$ which will give us the expected length of a chain. Then

$$E[\sum_i X_i] = \sum_i E[X_i] = \sum_i \Pr[X_i = 1] \leq \sum_i \frac{m}{m^2} = \frac{n}{m}$$

The inequality follows since the probability that x conflicts with x_i can be calculated as

$$\sum_{j=0}^{m-1} \Pr[h(X_i) = j \wedge h(x) = j] \leq \sum_{j=0}^{m-1} \frac{1}{m^2} = \frac{1}{m}$$

5. Suppose Google Maps decided to not only display the fastest route but also the second fastest route. The second fastest route may also take the same time butit will be distinct from the fastest route. Assume that you have a graph of the road network where edges represent roads and vertices represent junctions and the edges have weights associated representing the travel times. Design an efficient algorithm to find the two fastest routes from a given starting point to all possible destinations.

Your solution must have a rigorous justification why the output is correct, i.e., that the second shortest path is indeed so. The running time must be polynomial time and more credit will be given for faster algorithm. **(15)**

Denote the shortest path by P_1 and the second shortest path by P_2. Since P_1 and P_2 must differ by at least one edge, we can run any of the shortest path algorithm on $G - e_i$ where $e_i \in P_1$. Output the shortest paths among all these outputs. The running time is therefore k times the shortest path algorithm (either Dijkstra or BF) where $k = |P_1|$ and clearly $k < n$. The correctness follows from the observation that all the edges in P_2 must be present in in $G - e'$ where $e' \in P_1 - P_2$. It must be the shortest path as it is distinct from P_1.

This could cost us $|E|$ times the complexity of the SSSP algorithm - we can improve the efficiency by observing that any SSSP algorithm produces a *shortest path tree* T since every vertex has a unique predecessor (and no cycles). So, the total number of edges that we need to consider is $O(|V|)$. For any shortest path (s, v) from a fixed source s, denote by $D_e(s, v)$ as the shortest path distance to v in the graph $G - \{e\}$ for $e \in T$. Then the second shortest path from s to v is

$$D'(s, v) = \min_{e \in P(s,v)} D(s, v) \quad P(s, v) \text{ is the shortest path between } s, v$$

The total number of min operations over all v is $O(|V|^2)$ and the total cost is $O(|V| \cdot$ SSSP cost $+|V|^2)$ where SSSP algorithm may be Dijkstra or BF. If we can use Dijkstra, then it is $O(|V||E|\log|V|)$ assuming that the graph is connected.

Answers that lacked clarity and formal correctness proof were penalized heavily.

Another alternative is to modify any of the existing shortest path algorithm but that will require a separate proof of correctness. Many answers tried to modify Dijkstra but did not provide convincing proofs why that would work correctly. The proof for Dijkstra's algorithm was done quite elaborately in the lectures. Here is one for BF.

When we run the shortcutting step, we will maintain both the minimum and the second minimum estimates - $\Delta_1(v), \Delta_2(v)$ for each vertex $v \neq s$. When these are updated during a *relaxation step*, we will retain the smallest and the second smallest estimates. In the case of BF, we will terminate after $n-1$ iterations and output the two smallest estimates as the shortest and second shortest paths.

To justify this, we observe that P_2 must come through one of the vertices u where $(u,v) \in E$ is a directed edge coming into v. The second shortest path P_2 would come in through one one of these vertices u and should either be (i) the shortest path to u or (ii) second shortest path to u. Otherwise the length of P_2 cannot be second shortest.

We can now apply induction on the lengths of the second shortest path, in the same way that we proved BF on the number of edges on the path.

However, it can produce paths with cycles as there are no checks for that.

6. Given an array $A = x_1x_2\ldots x_n$, where x_i corresponds to predicted stock values of some company on i-th day, we want to plan how long should one hold on to the stock that was purchased on the i-th day. Intuitively, we want to sell it off before the predicted value drops below the purchase value. Formally, we want to calculate for each x_i a value $V_i = \min_{j>i}\{j|x_j < x_i\}$. It is undefined if all elements to the right of x_i are larger than x_i. The problem is to compute V_i for all i for given array A. For example, for the array $[5, 3.5, 7, 1.6, 8]$, the $V_1 = 2, V_2 = 4, V_3 = 4, V_4 = U, V_5 = U$. Here U means undefined.
Design a linear time algorithm for the above problem. **(15)**
The basic observation is that if we consider the elements in the array $x_1, x_2\ldots x_j$ for which $V(i)$ is undefined, then they form an *increasing* subsequence $x_{i_1} < x_{i_2} < \ldots x_j$. When we consider the next element x_{j+1}, then we find x_{i_k} such that $x_{i_{k-1}} < x_{j+1} < x_{i_k}$ and for the subsequence $x_{i_k}, x_{i_{k+1}}\ldots$, the $V()$ value is x_{j+1}.

We will scan the numbers from left to right and maintain a stack S that is initialized to x_1. When we scan the next number x_{i+1} we will check if $x_{i+1} < x_i$ - if so, we will set $V_i = i+1$ and pop x_i from S. We will subsequently compare the top element T having index t with x_{i+1}. If $x_t > x_{i+1}$ then we pop x_t and output $V_t = i+1$ and repeat the process till $T < x_{i+1}$ and then push x_{i+1} into the stack. Since each element can be pushed into the stack exactly once and popped at most once, the running time is $O(n)$.

Proof of correctness

Suppose $x_{i_1}, x_{i_2}\ldots x_{i+k}$ is in the stack where $i_1 < i_2$... Then we claim that $x_{i_1} < x_{i_2}\ldots x_{i+k}$, i.e., it is an increasing subsequence of the initial sequence A. This can be proved by induction. Initially, there is only one element, so it is trivially true. Suppose it is true at stage i, then by the above algorithm, we keep popping elments that are larger than x_{i+1} and so when x_{i+1} is pushed, it is the largest element and the induction hypothesis holds.

Similarly, we can argue that the V_i are the nearest smaller element as We will scan the numbers from left to right and maintthe subsequence $x_{i_1}, x_{i_2}\ldots x_{i+k}$ are exactly those elements for which we haven't found a smaller element and therefore they form an increasing sub-sequence.

Marks were deducted for not giving a rigorous justification why the values returned were indeed correct

7. There is an $L \times H$ metal sheet which can be cut by a machine into two pieces either vertically or horizontally - note that the cut completely slices through and is not partial. Both L, H are integral and the cuts also happen along integral values. There are n rectangular patterns $\ell_i \times h_i,\ i \leq n$ (ℓ_i, h_i are also integral) where the i-th pattern has profit c_i - the others have 0 profit. Design an efficient algorithm to cut the sheet in a way so as to maximize the total profit. Analyze the algorithm in terms of L, H, n. **(15)**
Let $N(a,b)$ denote the maximum profit by cutting a $a \times b$ sheet (a, b denote the length and height respectively). Then the following recurrence follows

$$N(a,b) = \max\{\max_{i=1}^{j=a-1}(N(i,b) + N(a-i,b)), \max_{j=1}^{j=b-1}(N(a,j) + N(a-i,b-j))\}$$

which chooses the maximum value over all possible horizonal and vertical cuts. Of course the base cases correspond to $N(\ell_i, h_i) = c_i$ and $N(1,1) = 0$ if there is no profit associated. More precisely, the vertical cut the possibilities are 1, 2, ..$\lceil L/2 \rceil$ and the horizontal can be at 1, 2, .. $\lceil H/2 \rceil$(the others are symmetric). The table $L \times H$ will be filled and to fill each entry to need to choose maximum of the $L/2 + H/2$ possible cuts. The optimum for each piece is already available. The rectangles $l_i \times h_i,\ \ i \leq n$ are base cases (strictly, speaking not all of them are base cases as you may obtain higher profit by cutting further). The time is $O(L^2H + H^2L + n)$ and the space is $O(LH)$. The entries may be filled row-wise or column-wise.

Marks were deducted if recurrence was missing, or the table-filling was not discussed

8. Given a rectangular subdivision of the plane (see Figure) consisting of n rectangles, we want to build a data structure to support queries of the following kind
Given a location X (specified by its x-y coordinates), we want to output the rectangle containing X - in this case $\square d, e, f, g$.
You can assume that each rectangle is specified by its 4 corner points and has labels $r_1, r_2 \ldots r_n$. Your algorithm should not take more than $O(npolylog)$ space and the query time should be $O(\log^2 n)$. Also analyse the time to build the data structure. **(15)**

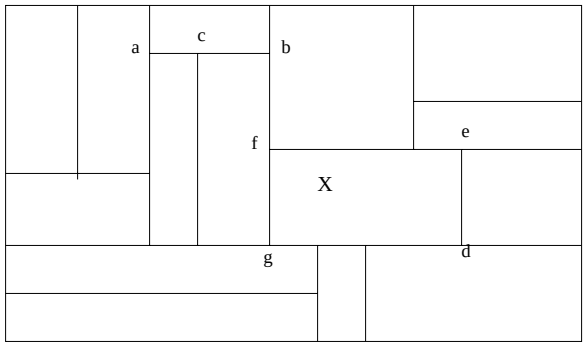

Figure 2: Not all the corner points are labelled and all the rectangular regions are axis parallel.

Using the horizontal edges, we can build a segment tree, where we store the segments in each node sorted along y direction. Our goal is to find the horizontal segment closest to the query point X which is above X (below will also work) since it is associated with a unique rectangle. To find it we will locate the query point X in the segment tree and do binary search in the y direction in all the nodes of the search path. This would yield $\log n$ segments and the closest among them will be the edge we are looking for.

The query time is $\log n \times O(\log n) = O(\log^2 n)$ and the space required is $O(n \log n)$ as per the segment tree bound. Sorting the segments in the horizontal direction can be done in $O(n \log n)$ time per level giving $On \log^2 n)$ total construction time. This can be reduced to $O(n \log n)$ using some clever use of integer sorting.